

Q.No:9

DS	Session	Tree 1	Question Information	Level 1	Challenge 69
----	---------	--------	----------------------	---------	--------------

Question description

A forest is an undirected graph without cycles (not necessarily connected).

Mohana and John are friends in Kerala, both of them have a forest with nodes numbered from 1 to n , and they would like to add edges to their forests such that:

- After adding edges, both of their graphs are still forests.
- They add the same edges. That is, if an edge (u,v) is added to Mohana's forest, then an edge (u,v) is added to John's forest, and vice versa.

Mohana and John want to know the maximum number of edges they can add, and which edges to add.

Constraints:

$1 \leq n \leq 105$,

$0 \leq m_1$

$m_2 < n$

$1 \leq u, v \leq n, u \neq v$

Input

The first line contains three integers n , m_1 and m_2 — the number of nodes and the number of initial edges in Mohana's forest and John's forest.

Each of the next m_1 lines contains two integers u and v — the edges in Mohana's forest.

Each of the next m_2 lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edges in John's forest.

Output

The first line contains only one integer h , the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3 2
5 4
2 1
4 3
4 3
1 4
```

EXPECTED OUTPUT

```
1
1 5
```

Test Case 2

INPUT (STDIN)

```
5 3 2
5 3
2 3
2 3
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 2
4 5
```

Code:

```
#include <stdio.h>

#define MAX 100005

int parent1[MAX], parent2[MAX];
int ans[MAX][2];
int cnt = 0;

int find1(int x)
{
    if(parent1[x] == x)
        return x;

    parent1[x] = find1(parent1[x]);
    return parent1[x];
}

int find2(int x)
{
    if(parent2[x] == x)
        return x;

    parent2[x] = find2(parent2[x]);
    return parent2[x];
}

void unite1(int a, int b)
{
    a = find1(a);
    b = find1(b);
    if(a != b)
        parent1[a] = b;
}

void unite2(int a, int b)
{
    a = find2(a);
    b = find2(b);

    if(a != b)
        parent2[a] = b;
}

int main()
{
    int n, m1, m2;
    int u, v;
    int i;
    scanf("%d %d %d", &n, &m1, &m2);
    for(i = 1; i <= n; i++)
    {
```

```
    parent1[i] = i;
    parent2[i] = i;
}
while(m1--)
{
    scanf("%d %d", &u, &v);
    unite1(u, v);
}
while(m2--)
{
    scanf("%d %d", &u, &v);
    unite2(u, v);
}
for(i = 2; i <= n; i++)
{
    if(find1(1) != find1(i) &&
       find2(1) != find2(i))
    {
        ans[cnt][0] = 1;
        ans[cnt][1] = i;
        cnt++;
        unite1(1, i);
        unite2(1, i);
    }
}
int a[MAX], b[MAX];
int ca = 0, cb = 0;

for(i = 2; i <= n; i++)
{
    if(find1(1) == find1(i) &&
       find2(1) != find2(i))
    {
        a[ca++] = i;
    }
    else if(find1(1) != find1(i) &&
            find2(1) == find2(i))
    {
        b[cb++] = i;
    }
}
int limit;
if(ca < cb)
    limit = ca;
else
    limit = cb;

for(i = 0; i < limit; i++)
{
    int x = a[i];
    int y = b[i];
```

```
if(x > y)
{
    int temp = x;
    x = y;
    y = temp;
}

ans[cnt][0] = x;
ans[cnt][1] = y;
cnt++;

unite1(x, y);
unite2(x, y);
}
printf("Output:\n");
printf("%d\n", cnt);

for(i = 0; i < cnt; i++)
{
    printf("%d %d\n", ans[i][0], ans[i][1]);
}
return 0;
}
```

Output:

Test_case 1:

```
5 3 2
5 4
2 1
4 3
4 3
1 4
Output:
1
1 5
```

Test_case 2:

```
5 3 2
5 3
2 3
2 3
2 3
2 4
Output:
2
1 2
4 5
```